

C01 — Rezervácie miestností: samostatný návod

Tento postup zvládnete aj vlastným tempom. Najprv si vo Visual Studiu vytvoríte malý program. Potom otvoríte pripravený projekt, nájdete chybu a doplníte jednu funkciu. Na cvičenie máte 90 minút. Programovať už viete; predchádzajúca prax v C# nie je nutná.

Ako s návodom pracovať

Chodte po poradí. V každej časti najprv vykonajte kroky a až potom si prečítajte odsek **Čo má vyjsť**. Ak sa výsledok líši, pozrite si nápovedu alebo časť **Čo ak niečo nejde?** na konci. Časy sú orientačné; keď sa zaseknete na prostredí, prejdite ďalej podľa uvedenej náhradnej cesty.

Na konci odovzdáte upravený projekt `starter/RoomBooking` a súbor `ODPOVEDE.md` tam, kam určí vyučujúci. `ODPOVEDE.md` si vytvoríte hneď na začiatku a **vypĺňate ho priebežne po každom kroku**, nie až na konci. Odpovede píšete vlastnými slovami. Ukážka v prezentácii nie je ďalšie zadanie; celý postup je v tomto návode.

Pred hodinou

V učebni má byť nainštalované Visual Studio 2026 aj VS Code. Na tomto cvičení používajte Visual Studio; v ďalších úlohách môžete pracovať aj vo VS Code. Ak šablónu pre konzolový projekt nevidíte, vo **Visual Studio Installer** skontrolujte workload **.NET desktop development**. Projekt používa .NET 10.

O študentský prístup ku GitHub Copilotu požiadajte, ak môžete, vopred na <https://github.com/settings/education/benefits>. Schválenie môže trvať dlhšie než jednu hodinu.

0. Začíname (0–5 min)

Rozbaľte ZIP na miesto, kde môžete ukladať súbory; nespúšťajte projekt priamo z otvoreného ZIP-u. V rozbalenom priečinku nájdete `C01-Rezervacie.slnx`, priečinok `starter` a súbor `ODPOVEDE-sablona.md`. Skopírujte `ODPOVEDE-sablona.md` do nového súboru `ODPOVEDE.md` v tom istom priečinku; doň budete zapisovať odpovede po každom kroku. Overte, že Visual Studio sa otvorí. Ak ešte nemáte aktivovaný študentský prístup ku Copilotu, požiadajte oň teraz na odkaze vyššie; na schválenie nečakajte. Súbor `.slnx` zatiaľ neotvárať.

1. Vlastný program „Hello World“ (5–13 min)

1. Vo Visual Studiu zvolíte **Create a new project**. Vyhľadajte **Console App** pre C#.
2. Projekt nazvite `Pozdrav`, zvolíte **.NET 10.0** a uložte ho **mimo rozbaleného ZIPu**.
3. Otvorte `Program.cs` a najprv spustíte pripravený „Hello, World!“ pomocou **Ctrl+F5**.
4. Zmeňte program tak, aby na dvoch riadkoch vypísal vaše meno a názov predmetu. Text aj dva príkazy `Console.WriteLine` napíšte sami. Spustite ho znovu.

Čo má vyjsť: V konzole uvidíte dva riadky, ktoré ste napísali vy, napríklad svoje meno a **Pokročilé objektové technológie**. `Program.cs` môže obsahovať príkazy priamo, bez triedy `Program` a metódy `Main`; ide o top-level statements. Tento prvý projekt sa do odovzdania nebalí; do `ODPOVEDE.md` zapíšete iba odpoveď do časti Vlastný program.

Ak po ôsmich minútach projekt ešte nespustíte, nechajte ho rozpracovaný a pokračujte. Požiadajte vyučujúceho o pomoc s prostredím, aby vám nezostalo málo času na hlavnú úlohu.

2. Spustenie pripraveného projektu (13–23 min)

Zatvorte projekt **Pozdrav** (**File** → **Close Solution**). Potom otvorte **C01-Rezervacie.slnx** z rozbaleného balíka (**File** → **Open** → **Project/Solution**). V **Solution Explorer** má byť projekt **RoomBooking**.

1. V **Program.cs** si prečítajte štyri pokusy a pred spustením si zapíšte, ktoré podľa vás uspejú. Ak jedna rezervácia končí o 10:00, ďalšia sa o 10:00 môže začať.
2. V hornej lište vyberte profil **Rezervácie - demo** a stlačte **Ctrl+F5**. Porovnajme výpis so svojou predpoveďou.
3. Vyberte profil **Rezervácie - kontroly** a spustite ho cez **Ctrl+F5**. Zapíšte si názvy neúspešných kontrol. Zatiaľ nič neopravujte.

Čo má vyjsť: Prvý pokus v A101 uspeje. Druhý by mal tiež uspieť, ale zatiaľ neuspeje. Pokus 09:30–10:30 v A101 je konflikt; rovnaký čas v B202 je povolený. Profil kontrol z deviatich kontrol hlási **šest' neúspechov**: dve kontroly dotyku susedných intervalov (v oboch poradiach) a štyri kontroly presunu. Na konci výpisu je **Neúspešné kontroly: 6** a proces môže skončiť s kódom 1. Neúspechy sú pripravené zámerne; nejde o pokazenú inštaláciu.

Do **ODPOVEDE.md** doplňte časť Predpoveď pred spustením.

3. Zorientujte sa v kóde (23–35 min)

Najprv bez Copilota otvorte **Program.cs**, **TimeRange.cs** a **ReservationService.cs**. Až potom si pozrite **Reservation.cs**, **IReservationStore.cs**, **InMemoryReservationStore.cs** a **SelfTests.cs**.

Načrtnite cestu jedného pokusu o rezerváciu od **Program** po **store.Add**. Ku každej triede si poznačte, za čo zodpovedá. Odpovedzte:

- Ktorá trieda vie, či sa intervaly prekrývajú?
- Ktorá trieda rozhoduje, či konflikt platí aj pre inú miestnosť?
- Ktorá trieda uchováva rezervácie?
- Aký je rozdiel medzi **ReservationStatus.Conflict** a **InvalidRange**?

Čo má vyjsť: Prekrýv zisťuje **TimeRange.Overlaps**. O tom, či ide o tú istú miestnosť, rozhoduje **ReservationService.HasConflict**. Rezervácie drží **InMemoryReservationStore**. Neplatný interval sa končí skôr, než sa začína, alebo v rovnakom okamihu.

Do **ODPOVEDE.md** doplňte časť Mapa kódu.

Nápoveda ku C# zápisu

record uchováva údaje a porovnáva ich podľa hodnôt. **IReservationStore** je rozhranie. **FirstOrDefault** vráti nájdenú rezerváciu alebo **null**. **Any** odpovie, či aspoň jedna položka spĺňa podmienku.

class ReservationService(IReservationStore store) je primary constructor: parameter **store** je dostupný vo všetkých metódach triedy, podobne ako súkromné pole. **Guid?** a **Reservation?** znamenajú, že hodnota môže byť **null**. Výkričník v **store.Find(id)!.Time** hovorí prekladaču „tu viem, že výsledok nie je **null**“. Zápis **x => ...** je lambda, teda krátka anonymná funkcia. **args is ["--self-test"]** testuje, či program dostal práve jeden argument s touto hodnotou, a **[]** vytvorí prázdny zoznam.

4. Nájdite chybu debuggerom (35–48 min)

1. Otvorte `TimeRange.cs`. Kliknite do ľavého okraja pri riadku `return` v metóde `Overlaps` alebo na ňom stlačte **F9**. Vznikne červený breakpoint.
2. Znovu spustíte profil **Rezervácie - demo** pomocou **F5**, aby sa program zastavil na breakpointe.
3. Program sa prvýkrát zastaví pri pokuse 10:00–11:00; pri prvom pokuse je úložisko prázdne a `Overlaps` sa nevolá. V okne **Locals** rozbaľte `this` a `other`. `this` je už uložená rezervácia 09:00–10:00, `other` je nový pokus 10:00–11:00. Sledujte `Start`, `End`, `other.Start` a `other.End`. Ak okno nevidíte, otvorte **Debug** → **Windows** → **Locals**; hodnotu uvidíte aj po podržaní myši nad názvom v kóde. Podľa potreby krokujte klávesom **F10**; beh zastavíte cez **Shift+F5**.
4. Napíšte, ktorá časť podmienky vyhodnotí dotyk na hranici ako konflikt a prečo to odporuje pravidlu [začiatok, koniec).

Čo má vyjsť: Intervaly 09:00–10:00 a 10:00–11:00 sa neprekrývajú. Spoločná hranica nemá vytvoriť konflikt.

Do `ODPOVEDE.md` si zapíšete hodnoty, ktoré ste v debuggeri videli.

Nápoveda k ladeniu 1

Pozrite sa na rovnosť `End == other.Start` (10:00 == 10:00). Ktorý operátor ju považuje za prekryv? Potom si predstavte rovnaké intervaly v opačnom poradí: vtedy platí `Start == other.End`. Aj túto rovnosť treba posúdiť.

Nápoveda k ladeniu 2

Pri poloootvorených intervaloch musia oba začiatky ležať **striktne pred** koncom druhého intervalu.

5. Opravte pravidlo prekryvu (48–58 min)

Opravte `Overlaps` čo najmenšou zmenou. Spustíte profil **Rezervácie - demo** a potom **Rezervácie - kontroly**. Overta oba prípady: dotyk na hranici je povolený a skutočný prekryv 09:30–10:30 je stále konflikt.

Čo má vyjsť: Demo vypíše `Success`, `Success`, `Conflict`, `Success`. V kontrolách prejdú obe kontroly susedných intervalov a zlyhávajú už len štyri kontroly metódy `Move`. Tá ešte nie je naprogramovaná; výnimka `NotImplementedException` je teraz očakávaná. Ak jedna z kontrol „susedné intervaly...” stále zlyháva, opravili ste iba jedno z dvoch porovnaní.

Do `ODPOVEDE.md` doplňte časť Chyba na hranici intervalov.

Ak chcete použiť Copilot, skúste: „Vysvetli podmienku `Overlaps` pre poloootvorené intervaly. Uveď protipríklad pre nesprávny operátor a test, ktorý zmenu overí.” Návrh porovnajte s hodnotami z debuggeru.

6. Naprogramujte presun rezervácie (58–80 min)

Teraz napíšete vlastný kód. Implementujte `ReservationService.Move(Guid id, TimeRange newTime)` podľa pravidiel:

1. Ak ID neexistuje, vráťte `NotFound` a nemeňte úložisko.
2. Ak je interval neplatný, vráťte `InvalidRange` a nemeňte úložisko.
3. Ak nový interval kolидуje s **inou** rezerváciou v rovnakej miestnosti, vráťte `Conflict` a nemeňte úložisko.
4. Inak aktualizujte čas rezervácie pri zachovaní ID, miestnosti a vlastníka; vráťte `Success` s aktualizovanou rezerváciou.

Najprv vyriešte body 1 a 2. Potom kontrolu konfliktu; môžete znovu použiť `HasConflict(room, time, exceptId)`. Nakoniec zmeňte uloženú rezerváciu. Presun na ten istý interval je povolený. Metóda `Book` ukazuje podobný postup.

Čo má vyjsť: Po dokončení prejde všetkých **deväť pripravených kontrol** a výpis končí vetou **Všetky kontroly prešli**. Pri zamietnutom presune zostane pôvodný čas uložený. Ak po desiatich minútach stále dostávate `NotImplementedException`, otvorte prvú nápovedu. Skontrolujte, že upravujete `starter/RoomBooking/ReservationService.cs`, nie iný súbor.

Do `ODPOVEDE.md` doplňte časť Presun rezervácie.

Nápoveda k presunu 1

Najprv použite `store.Find(id)`. Ak je výsledok `null`, nepokračujte. Potom skontrolujte `newTime.IsValid`.

Nápoveda k presunu 2

`Move` nedostáva miestnosť ako parameter; zistíte ju z nájdenej rezervácie ako `current.Room`. Pri `HasConflict` odovzdajte ID práve presúvanej rezervácie ako `exceptId`. Inak program nájde konflikt rezervácie so sebou samou.

Nápoveda k presunu 3

`record` možno skopírovať so zmenenou vlastnosťou pomocou `current with { Time = newTime }`. Až potom, keď všetky kontroly prejdú, zavolajte `store.Replace`.

7. Vlastný test a odovzdanie (80–90 min)

Do `SelfTests.cs` pridajte **jeden vlastný test**: presun na interval 12:00–12:00 musí vrátiť `InvalidRange` a pôvodná rezervácia musí zostať uložená. Skontrolujte obe podmienky, nielen návratový stav. Nový blok `Check(...)` vložte pred záverečný výpis výsledku. Ako vzor použite kontrolu „konfliktný presun ponechá pôvodný stav“: vytvorte `store`, rezervujte čas, zavolajte `Move` a porovnajte `store.Find(id)!.Time` s pôvodným časom.

Ak ste hotoví skôr, pridajte druhý test: zarezervujte `A101` na 09:00–10:00 a potom skúste `a101` (malým písmenom) na 09:30–10:30. Výsledok musí byť `Conflict`, pretože názov miestnosti nemá rozlišovať veľké a malé písmená.

Spustíte profil kontrol. Po pridaní jedného vlastného testu by malo prejsť **desať kontrol**; druhý dobrovoľný test počet zvýši na jedenásť. Dokončíte `ODPOVEDE.md` časťou Vlastný test a AI a skontrolujte, že sú vyplnené všetky časti. Odovzdajte priečinok `starter/RoomBooking` bez podpriechinkov `bin` a `obj` a súbor `ODPOVEDE.md` tam, kam určí vyučujúci.

Čo ak niečo nejde?

- **Nevidíte Console App pri vlastnom projekte:** vo Visual Studio Installer skontrolujte workload **.NET desktop development**. Ak je .NET 10 nainštalovaný, pokračujte dodaným projektom a povedzte to vyučujúcemu.
- **Spustenie hlási chýbajúce .NET 10:** overte inštaláciu SDK a vybraný workload vo Visual Studio Installer. Ak inštaláciu nestihnute, pracujte vo dvojici na pripravenom počítači.
- **Nevidíte profily:** otvorte súbor `.slnx`, nie iba jednotlivý `.cs` súbor; projekt má `Properties/launchSettings.json`.
- **Kontroly vidno ako obyčajný konzolový výpis:** áno, sú to pripravené kontroly v tej istej aplikácii, nie projekt pre Test Explorer.

- **NotImplementedException:** metóda **Move** ešte čaká na implementáciu; pred touto úlohou je to očakávané.
- **Po oprave hranice zostáva päť alebo šesť neúspechov:** jedna alebo obe kontroly „susedné intervaly...” stále zlyhávajú. Skontrolujte oba operátory v **Overlaps**; jedna kontrola skúša intervaly v poradí 09–10, 10–11 a druhá v opačnom poradí.
- **V konzole vidíte ? namiesto diakritiky alebo šípky:** skontrolujte, či je na začiatku **Program.cs** riadok **Console.OutputEncoding = System.Text.Encoding.UTF8;**. Výsledky kontrol to neovplyvňuje.
- **Copilot navrhol veľký prepis:** vyžiadajte si menšiu zmenu, skontrolujte zmenené riadky a porovnajte výsledok s očakávanými kontrolami.
- **Kontrola po vašej zmene stále ukazuje starý výsledok:** zastavte bežiaci program, uložte všetky súbory a vo Visual Studiu použite **Build** → **Rebuild Solution**. Potom znovu spustíte správny profil.